

11. MAP ADT

Map concepts

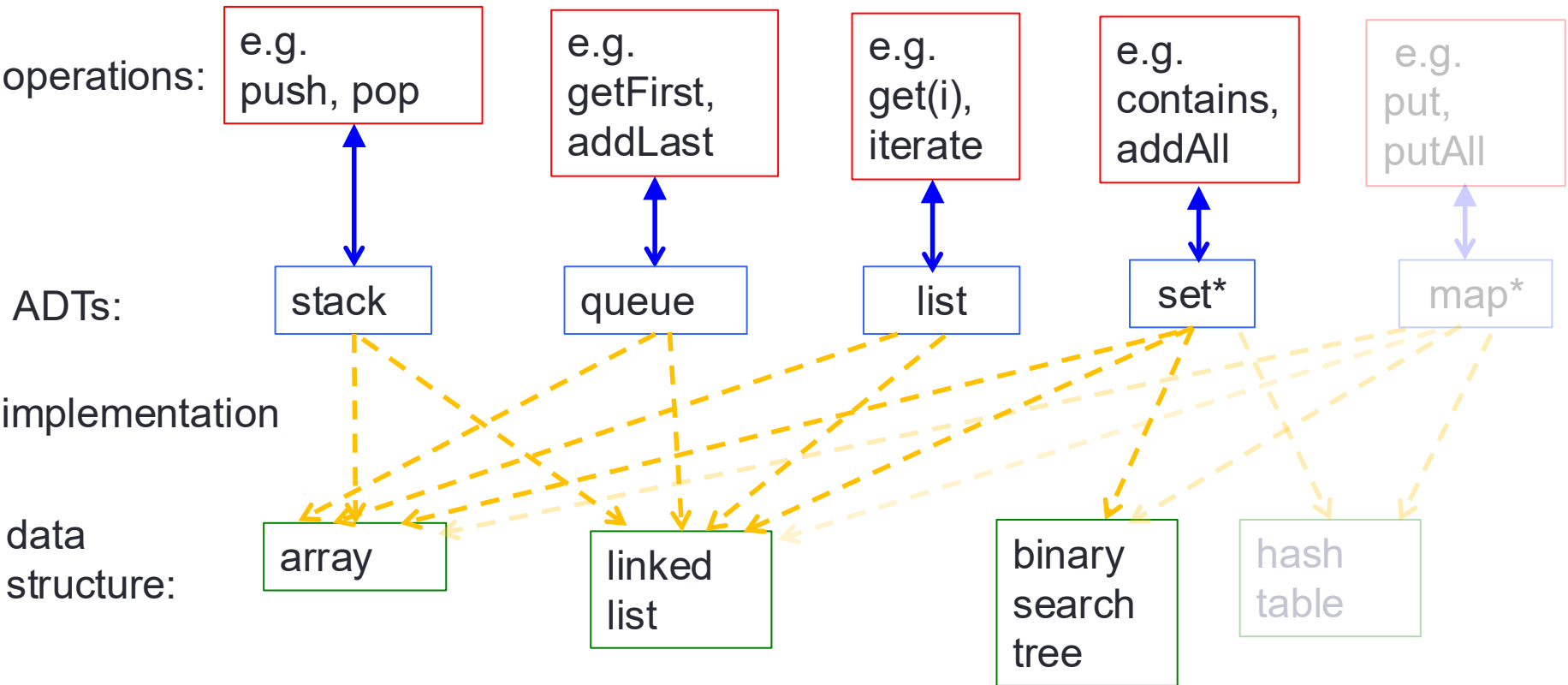
Map applications

A map ADT: requirements, contract.

Implementations of maps: using key-indexed arrays, entry arrays, linked-lists, BSTs

Maps in the Java class library

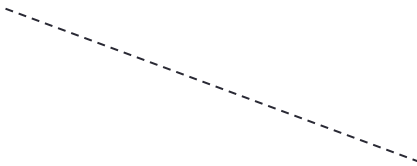
Update on diagram - what is missing?



* + special boolean/value array implementations when entries/keys are small integers

Map concepts (1)

- A **map** is a collection of entries, in no fixed order, in which all entries have distinct keys.
- Each **entry** is a tuple that consists of a **key field** and a **value field**.



More generally, an entry is a tuple consisting of one *or more* key fields and one *or more* value fields.

Example: maps

- Map with (letter, value) entries:
- Maps with (country, currency) entries:

Roman =

<u>letter</u>	value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

key field value field

EU =

<u>country</u>	currency
DE	euro
FR	euro
IT	euro
DK	krone
ES	euro

NAFTA =

<u>country</u>	currency
CA	dollar
US	dollar
MX	peso

Map concepts (2)

- The **size** (or **cardinality** 集合的大小) of a map m is the number of entries in m . This is written $\#m$. E.g.:
 $\#NAFTA = 3$
- An **empty** map has no entries.
- We can **lookup** a map for a given key, to obtain the corresponding value (if any). E.g.:
looking up *NAFTA* for US gives “dollar”
looking up *NAFTA* for UK gives nothing

Map concepts (3)

- We can **overlay** map m_1 by map m_2 . The result contains all entries from both m_1 and m_2 , except that where both maps contain entries with the same key, the result contains the entry from m_2 only. E.g.:

count1 =

<u>state</u>	winner
New York	Gore
Texas	Bush
Florida	Gore

count2 =

<u>state</u>	winner
Ohio	Bush
Florida	Bush

Result of overlaying
count1 by *count2* =

<u>state</u>	winner
New York	Gore
Texas	Bush
Ohio	Bush
Florida	Bush

Map applications (1)

- Relational database (RDB):
 - An RDB table is a collection of tuples, in no particular order.
 - If an RDB table has a key field, all tuples must have distinct keys.
So an RDB relation with a key field is in fact a map.

Map applications (2)

- Phone book:
 - Each entry consists of a name, address, and phone number.
 - Several entries may share the same name or the same address, but each entry must have a unique name-and-address combination.
 - The entries could be in any order. (They are typically sorted by name and address, to make lookup fast, but that is not essential.)
 - Thus, a phone book is a map in which the key field is the name-and-address combination.

Map ADT: requirements

- Requirements:
 - 1) It must be possible to make a map empty.
 - 2) It must be possible to test whether a map is empty.
 - 3) It must be possible to test whether a map contains an entry with a given key.
 - 4) It must be possible to look up the value corresponding to a given key in a map.
 - 5) It must be possible to add a new entry to a map or to replace an existing entry.
 - 6) It must be possible to remove an entry from a map, given its key.
 - 7) It must be possible to test whether two maps are equal.
 - 8) It must be possible to compute the overlay of two maps.
 - 9) It must be possible to traverse a map.

Map ADT: contract (1)

- Possible contract for homogeneous maps:

```
public interface Map<K, V> {  
    // Each Map<K, V> object is a homogeneous map  
    // whose keys and values are of types K and V  
    // respectively.  
    ////////////////// Accessors //////////////////  
    public boolean isEmpty ();  
    // Return true if and only if this map is empty.  
    public int size ();  
    // Return the size of this map.
```

Map ADT: contract (2)

- Possible contract (*continued*):

```
public V get (K key);  
// Return the value in the entry with key in this map,  
// or null if there is no such entry.  
  
public boolean equals (Map<K,V> that);  
// Return true if this map is equal to that.  
  
public Set<K> keySet ();  
// Return the set of all keys in this map.  
  
//////////////// Transformers //////////////////  
  
public void clear ();  
// Make this map empty.
```

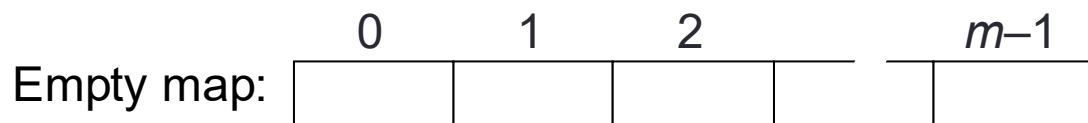
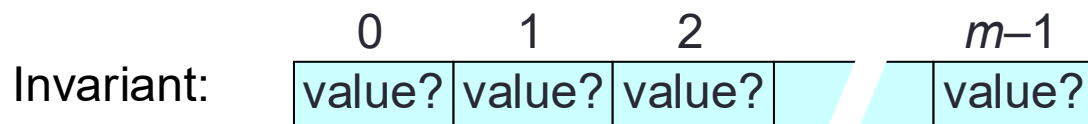
Map ADT: contract (3)

- Possible contract (*continued*):

```
public V remove (K key);  
// Remove the entry with key (if any) from this map.  
// Return the value in that entry, or null if there was  
// no such entry.  
  
public V put (K key, V val);  
// Add the entry (key, val) to this map, replacing any  
// existing entry whose key is key. Return the value in  
// that entry, or null if there was no such entry.  
  
public void putAll (Map<K, V> that);  
// Overlay this map with that, i.e., add all entries of  
// that to this map, replacing any existing entries  
// with the same keys.  
  
}
```

Implementation of small-integer-key maps using key-indexed arrays (1)

- If the keys are known to be small **integers**, in the range $0 \dots m-1$, represent the map by:
 - an array *vals* of length m , such that $vals[k]$ contains a value v if and only if (k, v) is a entry of the map.



Implementation of small-integer-key maps using key-indexed arrays (2)

Illustration
($m = 20$):

<u>code</u>	module
01	CS1
02	CS2
10	DB
11	OOP
12	ADS
14	OS
16	HCI

is represented by

0	1	2	3	10	11	12	13	14	15	16	17	18	19
	CS1	CS2		DB	OOP	ADS		OS		HCI			

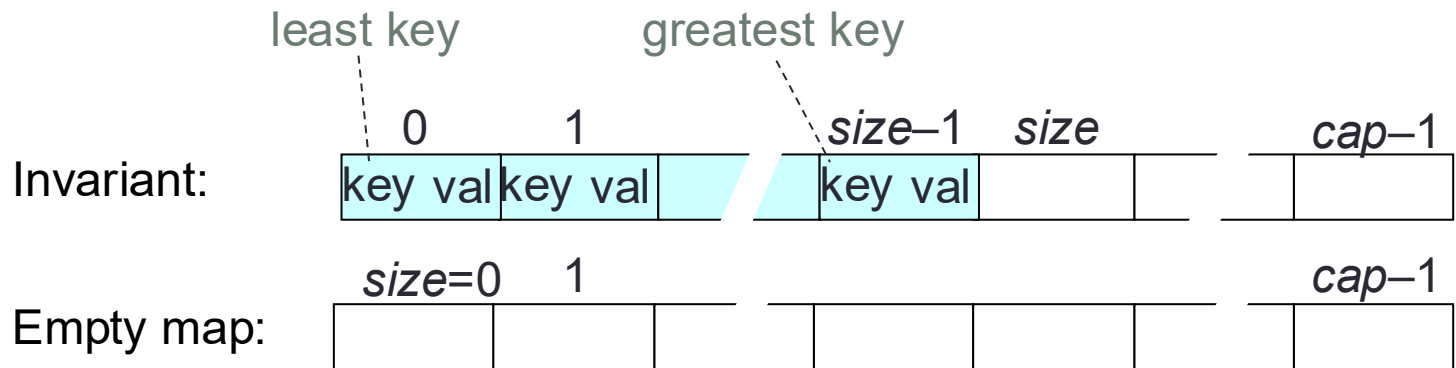
Implementation of small-integer-key maps using key-indexed arrays (3)

- Summary of algorithms and time complexities
(m is size of array, can assume second Map has at most this size, as keys will be within range).

Operation	Algorithm	Time complexity
get	inspect array element	$O(1)$
remove	make array element null	$O(1)$
put	update array element	$O(1)$
putAll	pairwise update all array elements	$O(m)$
equals	pairwise equality test	$O(m)$

Implementation of maps using entry arrays (1)

- Represent a **bounded** map ($\text{size} \leq \text{cap}$) by:
 - a variable *size*
 - an array *entries* of length *cap*, containing the map entries in *entries*[0...*size*−1], which is kept sorted by key.



Implementation of maps using entry arrays (2)

Illustration
(*cap* = 9):

<u>element</u>	number
He	2
Ne	10
Ar	18
Kr	36
Xe	54
Rn	86

is represented by

0	1	2	3	4	5	<i>size=6</i>	7	8
Ar 18	He 2	Kr 36	Ne 10	Rn 86	Xe 54			

Implementation of maps using entry arrays (3)

- Summary of algorithms and time complexities:

Operation	Algorithm	Time complexity
get	binary search	$O(\log n)$
remove	binary search, then array deletion	$O(n)$
put	binary search, then array insertion	$O(n)$
putAll	variant of array merge	$O(n+n')$
equals	pairwise equality test	$O(n')$

where n' is the size of the 2nd map

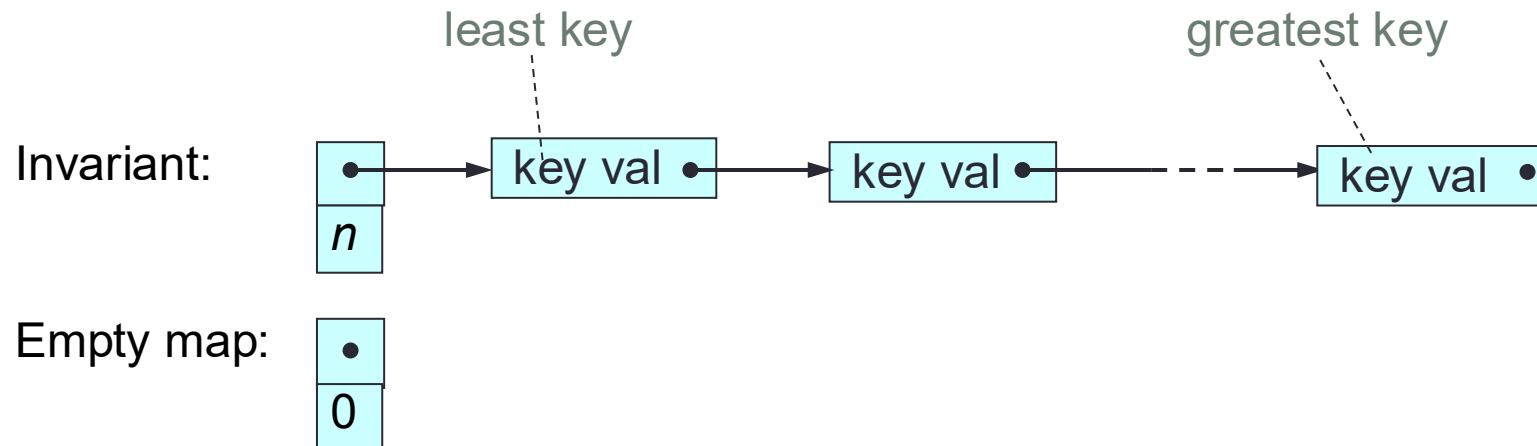
Implementation on Moodle

- Implementation of ArrayMap on Moodle. Note
- To instantiate an array of **Entries of generic type** have to use wildcards.

```
Entries = new (Entry<K,V>[]) new Entry<?,?>[maxCard]
```

Implementation of maps using SLLs (1)

- Represent an (unbounded) map by:
 - a variable *size*
 - an SLL, containing one entry per node, which is kept sorted by key.

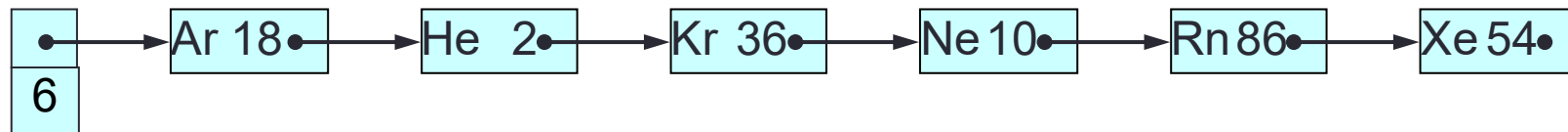


Implementation of maps using SLLs (2)

Illustration:

<u>element</u>	number
He	2
Ne	10
Ar	18
Kr	36
Xe	54
Rn	86

is represented by



Implementation of maps using SLLs (3)

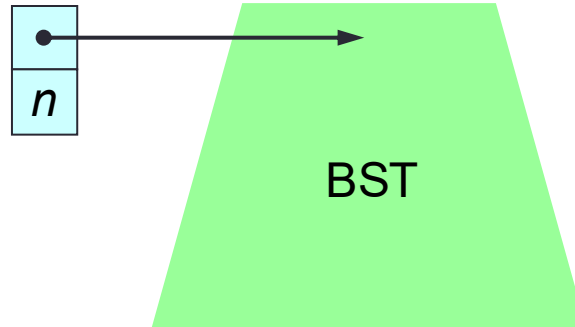
- Summary of algorithms and time complexities:

Operation	Algorithm	Time complexity
get	SLL linear search	$O(n)$
remove	SLL linear search, then deletion	$O(n)$
put	SLL linear search, then insertion	$O(n)$
putAll	variant of SLL merge	$O(n+n')$
equals	pairwise equality test	$O(n')$

Implementation of maps using BSTs (1)

- Represent an (unbounded) map by:
 - a variable *size*
 - a BST, containing one entry per node, in which the entries are ordered by their keys.

Invariant:



Empty map:

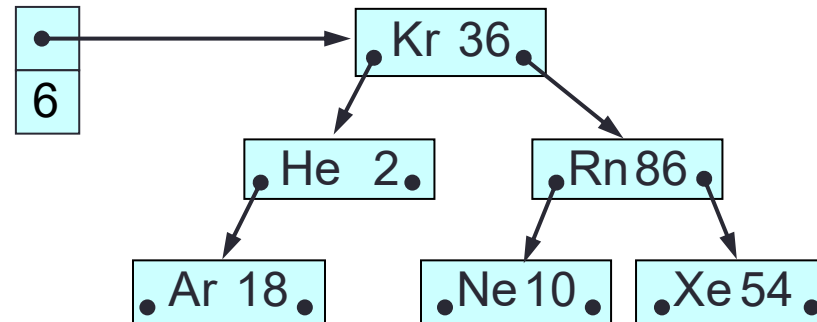


Implementation of maps using BSTs (2)

Illustration:

<u>element</u>	number
He	2
Ne	10
Ar	18
Kr	36
Xe	54
Rn	86

could be represented by



Implementation of maps using BSTs (3)

- Summary of algorithms and time complexities:

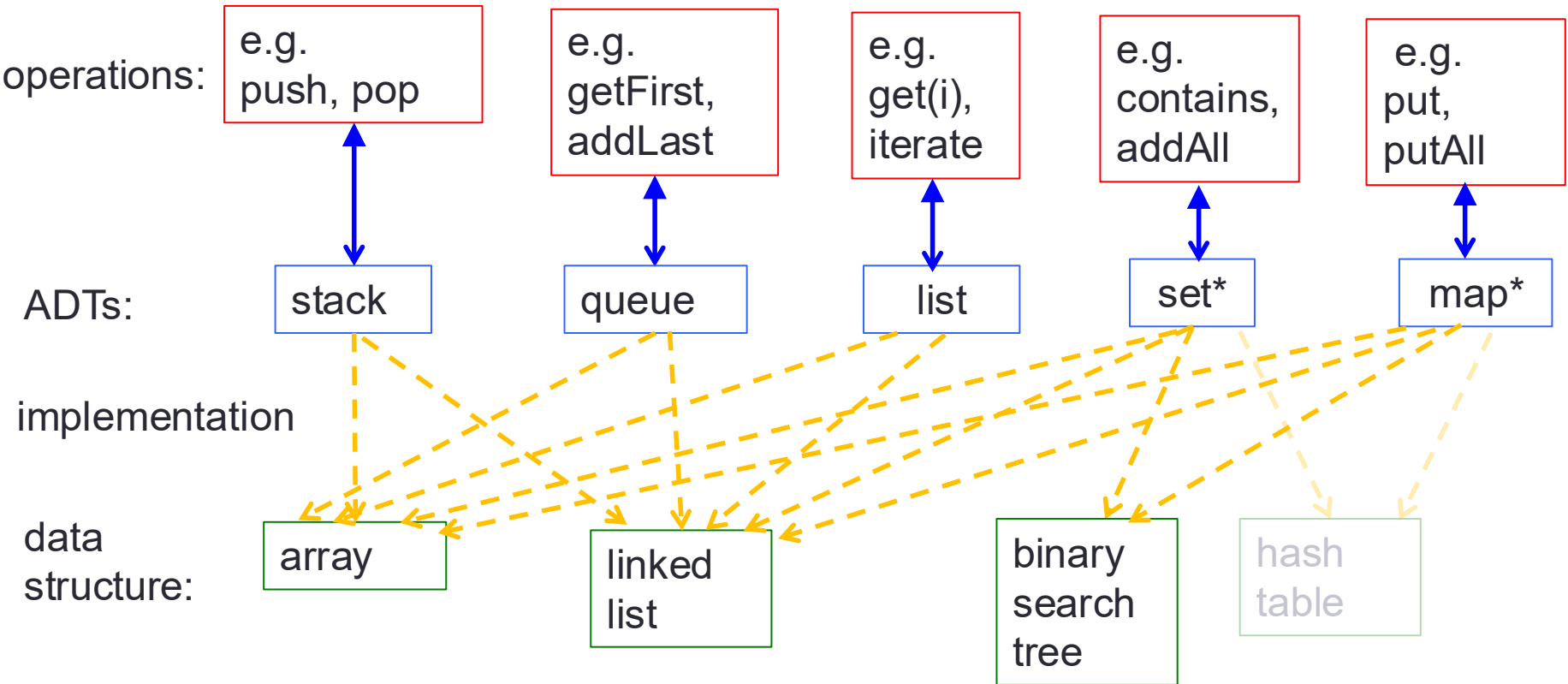
Operation	Algorithm	Worst case time complexity	
		balanced	unbalanced
get	BST search	$O(\log n)$	$O(n)$
remove	BST deletion	$O(\log n)$	$O(n)$
put	BST insertion	$O(\log n)$	$O(n)$
putAll	BST merge	$O(n' \log(n+n'))^*$	$O(n'n)$
equals	traversal of 2 nd set combined with search of 1 st BST	$O(n' \log n)$	$O(n'n)$

* Remember the BST grows as more elements added, so could have size up to $n+n'$:

Summary of map implementations

Operation	Key-indexed array representation	Entry array representation	SLL representation	BST representation balanced unbalanced	
get	$O(1)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$
remove	$O(1)$	$O(n)$	$O(n)$	$O(\log n)$	$O(n)$
put	$O(1)$	$O(n)$	$O(n)$	$O(\log n)$	$O(n)$
putAll	$O(m)$	$O(n+n')$	$O(n+n')$	$O(n'\log(n+n'))$	$O(n'n)$
equals	$O(m)$	$O(n')$	$O(n')$	$O(n'\log n)$	$O(n'n)$

Update on diagram - what is missing?



* + special boolean/value array implementations when entries/keys are small integers

Maps in the Java class library

- The library interface `java.util.Map<K, V>` is similar to the above interface `Map<K, V>`.
- The library class `java.util.TreeMap<K, V>` implements `java.util.Map<K, V>`, representing each map by a search-tree (actually a red-black tree).
- The library class `java.util.HashMap<K, V>` implements `java.util.Map<K, V>`, representing each map by a hash-table (§12).

Example: mobile phone contacts (1)

- Consider a simple mobile phone equipped with a keypad, display, and non-volatile memory.
- The memory contains a table of contacts. Each entry consists of a contact's name and phone number. All names must be distinct.
- The mobile phone enables the user to:
 - add a new contact, by entering the contact's name and phone number
 - display the contacts' names, one by one
 - make a call to the currently displayed contact
 - remove the currently displayed contact.

See Moodle for outline code